25. Paged Attention

28 March 2026 12:49

Require sufficient batching of requests at a time for high throughput serving of LLMs.

existing systems → KV cache memory for each request is huge and grows and shrinks dynamically

inefficient management leads to significant wastage through fragmentation, redundant duplication, limiting the batch size

Paged Attention → inspired by { classical virtual memory
paging techniques in operating systems

vLLM → an LLM serving system that achieves (1) near zero waste in KV cache memory
(2) flexible sharing of KV cache within and across requests to further reduce memory usage
↳ improves the throughput of LLM's by 2-4x with same level of latency compared to SOTA systems
↳ improvement is more pronounced with longer sequences, larger models and more complex decoding algorithms

KV cache
↳ helps with loading overhead
↳ Avoids computing past attention states

KV cache is per request
when the request ends its KV cache is no longer needed
⇒ the memory is released back

} Memory is not static
It is allocated & deallocated simultaneously

Increasing the throughput and hence reducing the cost per request

Auto regressive transformers → generate tokens one at a time
→ This loop until end token makes the workload memory bound under utilizing GPUs and limiting the serving throughput

Improve throughput → batching
(but memory space for each request should be efficiently managed)

Minimal system picture

User request → arrives at server
→ gets scheduled (possibly batched)
→ processed step by step
→ produces tokens over time
→ finishes → response sent

KV cache is tied to a single request's generation process

Life cycle
→ for one request
1. Request starts
→ KV cache created
II. Model generates tokens step by step
→ KV cache grows

Serving throughput

Improve throughput → batching
(but memory space for each request should be efficiently managed)

existing LLM serving until then → inefficient management of KV cache memory
↓
because they store the KV cache of a request in contiguous memory space

Most DL frameworks need tensors to be stored in contiguous memory

KV cache has unique characteristics : it dynamically grows and shrinks over time as the model generates new tokens, and its lifetime and length are not known apriori

existing systems suffer from internal and external memory fragmentation

To store KV cache in contiguous memory, they pre allocate a contiguous chunk of memory with the requests maximum length
- → this can result in severe internal fragmentation, since the request's actual length can be very low than its maximum
- → it would still be an issue even if the length is known in advance
  pre allocation is inefficient because the entire chunk is reserved, other shorter requests can't utilize the part of the chunk that is currently unused
- → external memory fragmentation

existing systems cannot exploit the opportunities for memory sharing

Paged attention inspired from virtual memory with paging
- → divides the KV cache of request into blocks, each of which contains the attention keys and values of a fixed no of tokens
  - → are not necessarily stored in a contiguous space

KV cache can be managed in a more flexible way

---

i) Model generates tokens step by step
→ KV cache grows
ii) Request finishes
→ KV cache is freed

KV caching is not tied to batching

not shared by default across requests

It exists only for that request's generation

Request ↔ One user's prompt → response
Batch ↔ System grouping of multiple requests for efficiency
KV cache ↔ Memory tied to each individual request

Inside a Batch
Each request has its own KV cache
These caches i) grow independently
ii) finish at different times
iii) are freed independently
Batch groups & computes but does not merge memory

think of KV blocks as pages, tokens as bytes and requests as processes

→ internal fragmentation alleviated by using relatively small blocks and allocating them on demand

↔ External fragmentation is also fixed as all blocks have same size

→ Enables memory sharing at the granularity of a block, across different sequences associated with the same requests or across the diff requests

→ System + Objective

Throughput vs cost per request
Memory bound nature of decoding
Batching improves utilization

Stored in contiguous memory → critical constraint
pre allocate max length → allocation strategy
Dynamic growth + unknown length → core mismatch
Memory dominates batch size → System bottleneck

vLLM → a high throughput distributed LLM serving engine

Once trained LLMs are often deployed as a conditional generation service

request to LLM (provides a list of input prompt tokens)
↓
LLM generates output tokens (paper refers to concatenation of prompt & output tokens as sequence)

→ Autoregressive + KV cache
Sequential generation
KV cach stores past states
Only new K,V computed per step
KV depends on full prefix

LLM generates tokens one by one, the generation process of each new token depends on all the previous tokens in that sequence, specifically their key and value vectors
often cached (KV cache)

→ Two phases of LLM serving
prompt phase = parallelizable
Generation phase = Sequential
Data dependency blocks parallelism

The prompt phase –
whole user prompt $(x_1 \ldots x_n)$ as input
Computes the probability of first new token $P(X_{n+1} | x_1 \ldots x_n)$
During this process, also generates Key vectors $K_1 K_2 \ldots K_n$
Value vectors $V_1 V_2 \ldots V_n$

→ Memory observations
weights → static
KV cache = dynamic
KV cache dominates runtime memory behaviour

prompt tokens are all known, the computation can be parallelized
↓
this can be done to efficiently use parallelism on GPUs

→ Fragmentation
Internal fragmentation from over allocation

The auto regressive generation phase Generates remaining new tokens sequentially

At iteration t token x_{n+t} becomes input

free memory exists

At iteration t token x_{n+t} becomes input

$P(x_{n+t+1} | x_1 \ldots x_{n+t})$ with the Key vectors $K_1 \ldots K_{n+t}$ and Value vectors $V_1 \ldots V_{n+t}$

completes when sequence
- reaches maximum length (specified by users or limited by LLMs)
- or when an <EOS> token is emitted

Key & value vectors from 1 to $n+t-1$ are cached at previous iterations
Only new K_{n+t} & V_{n+t} are computed at this iteration

this computation at different iterations cannot be parallelised due to Autoregressive nature
(data dependency & often uses matrix multiplication, which is less efficient)

This phase severely underutilizes GPU computation and becomes memory bound, being responsible for most portion of latency of a single request

Batching Techniques

batch multiple requests
(reqs share same model weights, the overhead of moving wts is amortised across requests in a batch)

large batch → computational overhead

Batching non trivial → (i) requests may come at different times
naive strategy → cause delays
(ii) requests may have vastly different input and output lengths
naive strategy might use padding to equalize length wasting resources

fine grained batching mechanisms
↑
New techniques operate at iteration level instead of request level

After each iteration, completed requests are removed from batch and new ones are added

free memory exists but cannot be used because it is not contiguous

Internal fragmentation from over allocation

External fragmentation from unusable gaps

→ Batching Limitations
- Waiting delays
- padding inefficiency
- Iteration level scheduling improves this

→ Fine grained batching
Iteration level scheduling vs batch level
Requests can enter/leave dynamically
Reduces waiting + padding inefficiency

→ Core bottleneck
GPU memory capacity limits batching
KV cache dominates memory
System is memory bound

→ KV cache challenges
- Large KV size per request
- Unknown sequence length
- Complex decoding increases memory complexity

→ Fragmentation
Internal → over allocation
External unusable free memory

happens when memory is free but not usable due to size/contiguity mismatch

→ High level Paged attention idea
Break KV cache into blocks
Non contiguous physical storage

After each iteration, completed requests are removed from batch and new ones are added

A new req can be processed after waiting an iteration but not batch process

Fine grained batching reduces the waste of compute, enables flexible batching of requests

But still the no of requests that can be batched together is still constrained by GPU memory capacity

Otherwords they say serving system's throughput is memory bound particularly the space allocated to store KV cache

Address Large KV cache, Complex decoding Algorithms, scheduling for unknown input & output lengths

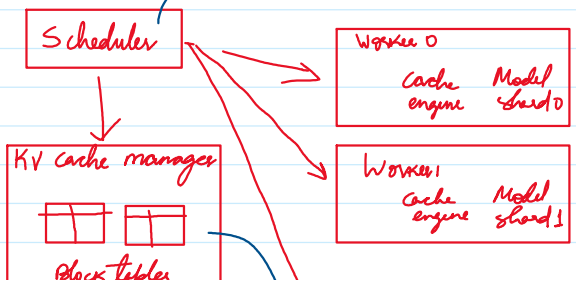
- Size grows quickly with the no of requests
- KV cache can be shared for cases like when users require sampling from 1 input
- for auto regressive generation KV cache should remain unshared

Chunk pre allocation

Existing systems then had three primary sources of memory wastes

(i) reserved slots for future tokens
(ii) internal fragmentation due to overprovision for maximum potential sequence lengths
(iii) external fragmentation from the memory allocator like buddy allocator

VLLM System Overview



Break KV cache into blocks

Non contiguous physical storage

Logical vs physical separation

Block table = Mapping

→ KV cache life cycle
filled left → right
Grows dynamically

KV cache is per request sequence

multiple sequences may share parts

Physical vs Logical Memory

It's about view vs reality

Logical → How the system thinks memory is arranged (contiguous view)

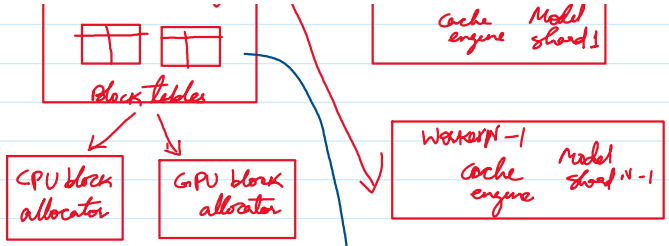
Physical → Actual GPU memory locations (non-contiguous)

Logical blocks → appear ordered

Physical blocks → allocated independently not guaranteed contiguous

Attention is computed block by block then Aggregated

* Block table = translation layer
Logical blocks ≠ physical layout
Allocation happens on demand



effectively manage the KV cache in a pr

Manages the physical KV cache memory on the GPU workers through the instructions sent by the centralized scheduler

→ Paged attention allows storing continuous keys & values in a non contiguous space

↳ partitions the KV cache of each sequence into KV blocks

each contains key, value vectors for a fixed no of tokens

Key block $K_j = (K_{(j-1)B+1} \quad K_{jB})$

Value block $V_j = (V_{(j-1)B+1} \quad V_{jB})$

$$A_{ij} = \frac{\exp(q_i^T K_j / \sqrt{d})}{\sum_{t=1}^{\lfloor i/B \rfloor} \exp(q_i^T K_t / \sqrt{d})}, \quad O_i = \sum_{j=1}^{\lfloor i/B \rfloor} V_j A_{ij}^T$$

$A_{ij} = (a_{i,(j-1)B+1} \quad a_{i,jB})$ is row vector of attention score on j^{th} KV block

During attention computation, paged attention kernel identifies and fetches different KV blocks separately

In summary Paged Attention algorithm allows the KV blocks to be stored in non-contiguous physical memory

↓

Thus enables more flexible paged memory management in vLLM

KV cache manager is analogous to the virtual memory in operating systems

OS partitions memory into fixed-sized pages and maps user programs local pages to physical pages

Contiguous local pages can correspond to non-contiguous physical memory pages allowing user programs to access memory as though it was contiguous

Allocation happens on demand

No pre allocation of max-length

Uniform block size

Think as

logical = "array of KV states per token"

physical = "scattered chunks in GPU memory"

Uniform granularity removes allocation incompatibility

with fixed size blocks (physical memory)

All blocks → uniform size

Any free block can satisfy any allocation request (block level)

No size mismatch problem

No need for contiguous large chunks

But fixed size block introduce

Internal fragmentation (at block level)

Before (contiguous pre allocation) Now (fixed blocks)

Large internal fragmentation ⟷ Small internal fragmentation

External fragmentation → Eliminated

→ Decoding process

token by token generation + KV updates

Decoding = autoregressive generation phase

Contiguous local pages can correspond to non-contiguous physical memory pages allowing user programs to access memory as though it was contiguous

Physical memory space needs not to be fully reserved in advance, enabling the OS to dynamically allocate physical pages as needed

↓

vLLM uses this idea to manage KV cache in an LLM service

Enabled by paged attention, we organize the KV cache as fixed size KV blocks, like pages in a virtual memory

A requests KV cache → represented as series of logical KV blocks filled from left to right as new tokens and their KV cache are generated

The last KV blocks unfilled positions are reserved for future generations

On GPU workers, a block engine allocates a contiguous chunk of GPU DRAM and divides it into physical KV blocks (this is also done on CPU RAM for swapping)

KV block manager also maintains block tables → mapping btwn logical and physical KV blocks of each request

Each block table entry records the corresponding physical blocks of a logical block and the no of filled positions

Separating logical and physical KV blocks allows vLLM to dynamically grow the KV cache memory w/out reserving it for all positions in advance which eliminates most memory waste in existing systems

How vLLM executes Paged attention and Manages the memory during the decoding process of a single input sequence

↓

as in OS's virtual memory vLLM doesn't require reserving the memory for maximum possible generated sequence length initially

It reserves only the necessary KV blocks to accommodate the KV cache generated during the prompt computation

Parallel Sampling

Decoding = autoregressive generation phase

each step read KV cache
compute next token
append new KV

→ No pre allocation
allocate blocks as needed

→ Block behavior
Logical blocks grow left → right
Physical blocks allocated dynamically
Block table maps logical → physical

→ Filling behavior
If space exists in last block - fill
Else allocate new block

→ Memory life cycle
Blocks freed after request

→ Parallel sampling intuition
Same prompt → shared KV cache
Avoids duplication

→ Reference counting
Tracks how many sequences use a block
Freed when no longer used

→ Copy on write
Shared until modification → then copy

Parallel Sampling

In LLM based programming, an LLM generates multiple sampled outputs for a single input prompt
↓ user can choose a favorite output

one request includes multiple samples sharing the same input prompt
↓ allowing KV cache of the prompt to be shared as well
via its paged attention and paged memory management, vLLM can realize this sharing easily & save memory

A single physical block can be mapped to multiple logical blocks

vLLM implements a "Copy on write" Mechanism at the block granularity for the physical blocks that need modification by multiple sequences

Beam search relies on the beam width parameter 'k', which determines the no of top candidates retained at every step
↓
during decoding expands each candidate sequence in the beam by considering all possible tokens, computes their respective probabilities using the LLM, and returns the top-k most probable sequences out of k|V| sequences

Auto regressive generation per sequence is sequential

Across different sequences (in a batch), computation can run in parallel

KV computations can happen in parallel across sequences/tokens not within a single sequence step

Different sequences in a batch → processed in parallel on GPU

During one decoding step → multiple sequences compute their next token simultaneously

KV computation → parallel across sequences, not across time steps

for each decoding step
1. Read KV cache (via block table)
2. Compute attention (block-to-seq)
3. Generate token
4. Store new KV in block
5. Update mapping

Shared until modification → then copy

* Generation is still sequential

Parallelism comes from
processing multiple tokens within a block during attention
Not from generating multiple tokens at once

KV for new token is still computed sequentially

Parallelism is
attention computation over stored KV blocks

vLLM concatenates tokens across batched sequences in that iteration

Copy on write triggers
↳ When a sequence modifies a shared block

Reference counting → enables safe sharing

Copy on write → avoids unnecessary duplication

Block-level sharing → across sequences

Dynamic allocation → no pre reservation

* parallelism exists across sequences (batch dimension) and across tokens processed within attention computation, but not across time steps of a single sequence

Beam Search

Beamsearch + paged attention

Beam Search

Keeps top K candidate sequence at each step
Avoids full combinatorial explosion
Each step expands candidates, then keeps top K

KV cache sharing — Initial prefix shared across beams
Divergence leads to different continuations

Block level sharing intuition: Early blocks shared, later blocks diverge

Each of K beams expands into V possibilities
then top K are selected

→ All beams start identical
Divergence happens when different tokens are chosen

Think: Tree of sequences (beam search)
KV cache = path history
Shared prefix = shared memory blocks
Divergence = new blocks
Pruning = decrement of ref counts

Shared prefix: commonly, the LLM user provides a (long) description of the task including instructions and example inputs and outputs (system prompt)
→ can be tuned via prompt engineering to improve accuracy of the downstream tasks

for this type of cases where many user prompts share a prefix, LLM provider can store the KV cache on the prefix

In vLLM → reserving a set of physical blocks for a set of predefined shared prefixes

A user input prompt with the shared prefix can simply map its logical blocks to the cached physical blocks (with the last block marked copy on write)

The prompt phase computation only needs to execute on the user's task input

vLLM facilitates the simultaneous processing of requests with different decoding preferences

The LLM and its execution kernel only see a list of physical block ID's for each sequence and do not need to handle sharing patterns across sequences

First come First serve (FCFS) scheduling policy

LLM services → input prompt length (unknown)

Beamsearch + paged attention

① prefix sharing
All beams share early KV blocks

② Divergence
When beams choose different tokens
new blocks allocated

③ Reference counting
Tracks how many beams use each block

④ clean up
When a beam is pruned
→ its blocks ref counts decrease
→ freed if zero

→ Shared prefixes
Many requests share same prefix
precompute + reuse KV cache
Avoid recomputation

→ Preemption & scheduling
Capacity overflow → needs prioritization
FCFS policy
Sequence groups for shared memory

→ Two recovery strategies
Swapping (GPU → CPU)

First come First serve (FCFS) scheduling policy

LLM services → input prompt length (unknown)
output length (unknown) (dependent)

requests grow, outputs grow → vLLM can run out GPU's physical blocks to store the newly computed KV cache

(1) which blocks should vLLM evict?
(2) How to recover evicted blocks if needed again?

All or nothing eviction policy (because all the blocks of a sequence accessed together)

Multiple sequences within one request are going scheduled as a sequence group
↓
the sequences within this group are scheduled together owing to the possibility of memory sharing

Recovery of evicted block? 1.) Swapping → copy to a swap space → copy evicted blocks to CPU memory (vLLM includes a CPU block allocator)
↓
upon exhaustion of free blocks
↓
selects sequences to evict and transfer their KV cache
↓
preempt a seq, wait until it completes
↓
req, if completed, blocks are freed and brought back

both (2.) Recomputation → simply recompute the KV cache when the preempted sequences are rescheduled

performances depend on bandwidth between CPU RAM and GPU memory

Recomputation latency can be lower than original latency

No of blocks swapped to CPU RAM never exceeds the total physical blocks in the GPU RAM

Many LLM params exceed single GPU capacity → Hence Distributed execution

Memory manager capable of handling distributed memory

SPMD (single program multiple data)

Linear layers are partitioned to perform block wise matrix multiplication, the GPU's constantly synchronize intermediate results via an all reduce operation

Preemption
= pause some ongoing sequences for freeing memory

FCFS decides which requests to run
preemption decides
→ which active ones to pause when memory is insufficient

→ two recovery strategies
Swapping (GPU → CPU)
Recomputation

→ Swapping flow
Evict KV Blocks to CPU
Free GPU memory
Restore later

→ Distributed Execution
Scheduler sends block tables + tokens
Workers compute using KV cache
Synchronization via GPU ops

→ Kernel Optimizations
Fuse operations
Reduce kernel launches
Handle non contiguous memory

→ Block size tradeoff
Too small → poor GPU utilization
Too big → internal fragmentation

→ Recomputation v Swapping
↓ small blocks ↓ large blocks

→ Evaluation stuff
2-4x throughput
Better normalized latency

The GPU's constantly synchronize intermediate results via an all reduce operation

Even with model parallel execution
Each model shard still processes the same set of input tokens
thus requiring the KV cache for the same positions

A single KV cache manager within the centralized Scheduler

In each step the scheduler first prepares the message with input token ID's for each token in the request
as well as the block table for each request

The scheduler broadcasts this control message to the GPU workers

GPU workers start to execute the model with input token ID's

In attention layers, the GPU workers read the KV cache according to the block table in the control message

GPU workers synchronize, use all-reduce communication primitive without the coordination of the scheduler

↓ Then send the sampled tokens back

vLLM is an end to end serving system with a fast api frontend and a GPU based inference engine

General systems don't support these memory access patterns

GPU Kernels to optimize this (i) fused reshape and block write
KV cache are split into blocks
reshaped to a memory layout optimized for block read
Saved at positions specified by block table

(ii) Fusing block read and attention
→ adapt attention kernel in faster transformer* to read

preemption decides
→ which active ones to pause when memory is insufficient

Eviction is at sequence level
either evict entire sequence KV or none

Recomputation adds computation overhead

GPU's
execute kernels using provided memory mapping

Memory management is handled by
KV cache manager + Scheduler

Kernels are designed to
read non-contiguous KV blocks efficiently
minimize memory access overhead

* Virtual Memory might not help Compute bound workflows

Preemption = memory pressure handling

Swapping vs recomputation = bandwidth vs compute tradeoff

Block size = utilization vs fragmentation tradeoff

Kernel design = make non-contiguous memory efficient

Everything revolves around
KV cache memory as

(II) fusing block read and attention
→ adapt attention kernel in faster transformer* to read KV cache according to the block table and perform attention operations on the fly

(III) Fused Block Copy
Block copy operations issued by copy-on-write mechanism may operate on discontinuous blocks
We implement a kernel that batches the copy operations for different blocks into a single kernel launch

Three Methods to support various decoding algorithms

- fork: creates a new sequence from existing one
- append: adds new tokens to these sequences in every iteration
- free: free method deletes the sequence

Faster transformer
↳ a distributed inference engine highly optimized for latency

Orca
↳ SOTA LLM serving system optimized for throughput

Normalized latency: The mean of every request's end to end latency divided by its output length

Ablation studies

(1) Kernel Microbenchmark
extra overhead of accessing block table (but minimal effect)

(2) Impact of Block size

Overlapping recourses around KV cache memory as the bottle neck

final mental model

System loop
① requests arrive
② Scheduler batches + schedules
③ KV cache grows dynamically
④ Memory pressure occurs
⑤ System responds via
block allocation
Sharing
Pre emption
Swapping / recomputation

⑥ GPUs executes kernels efficiently block mapped KV

" (but " minimal effect)

② Impact of Block size

if small → vLLM may not fully utilize the GPU's parallelism for reading and processing the cache

if large → internal fragmentation increases

③ Recomputation & swapping

Swapping incurs excessive overhead with small block sizes

because smaller block sizes often result in numerous small data transfers between CPU & GPU

Overhead remains constant across different block sizes for recomputation

Recomputation → when block size is small

Swapping → vice versa